

Ficha Pedagógica de Repaso: Arquitectura y Diagnóstico de Memoria en Windows 11

Cuaderno Digital NotebookLM: "LSO 4to - 2do semestre"

Laboratorio de Sistemas Operativos — Tecnicatura en Informática Personal y Profesional

Introducción para el Generador de Audio / Podcast de NotebookLM

En esta segunda sesión de repaso abordamos el corazón dinámico del Sistema Operativo: **la gestión de memoria**. ¿Cómo logra Windows 11 ejecutar simultáneamente decenas de procesos con gigabytes de código sobre una memoria física finita? Analizamos la ilusión de la **Memoria Virtual**, la traducción por hardware mediante la **MMU**, el mecanismo de **Paginación en bloques de 4 KB**, el costo oculto de los **Fallos de Página Duros** y la lectura médica del **Monitor de Recursos** (**resmon.exe**).

1. La Brecha de Latencia en la Jerarquía de Memorias

El procesador opera a frecuencias de gigahercios (miles de millones de ciclos por segundo). Si cada ciclo de reloj del CPU equivaliera a **1 segundo** en la vida humana:

- **Registros del CPU (< 1 ns):** 1 segundo.
- **Caché L1 (1 ns):** 3 segundos.
- **Caché L3 (10-20 ns):** 40 segundos.
- **Memoria RAM Física (50-100 ns):** 4 a 6 minutos.
- **SSD NVMe (100 µs):** ~3 a 4 días.
- **Disco Rígido Mecánico HDD (10 ms):** ~4 a 6 meses.

[!IMPORTANT] **Lección Fundamental de Arquitectura:** El procesador nunca interactúa directamente con el disco para ejecutar instrucciones. Todo debe ser cargado previamente en la memoria RAM. Si la CPU debe esperar a que un dato viaje desde el almacenamiento secundario, se detiene durante millones de ciclos de reloj desperdiciados.

2. La Ilusión de la Memoria Virtual y el VAS

Cada proceso que se ejecuta en Windows 11 no ve la memoria física real; ve un **Espacio de Direcciones Virtuales (Virtual Address Space - VAS)** continuo, privado y aislado:

- **Aislamiento Total:** El Proceso A no puede leer ni escribir en la memoria del Proceso B. Si el Proceso A colapsa por una violación de acceso (*Access Violation* / puntero nulo), el resto del sistema sigue funcionando intacto.
- **División Arquitectónica Usuario / Núcleo:**
 - En sistemas de 64 bits (x64), el espacio de direcciones virtuales asciende teóricamente a 16 Exabytes (con 128 TB asignados para Modo Usuario y 128 TB para Modo Núcleo en Windows 11).

- En Modo Usuario (*Ring 3*), los programas no tienen acceso a los registros del hardware ni a la memoria del sistema operativo. Solo a través de llamadas al sistema (*System Calls*) pueden solicitar servicios al Modo Kernel (*Ring 0*).
- **El Rol de la MMU (*Memory Management Unit*):** Es el chip integrado en el procesador encargado de traducir, a la velocidad de la luz mediante tablas indexadas, cada dirección virtual a una dirección física real en la memoria RAM.

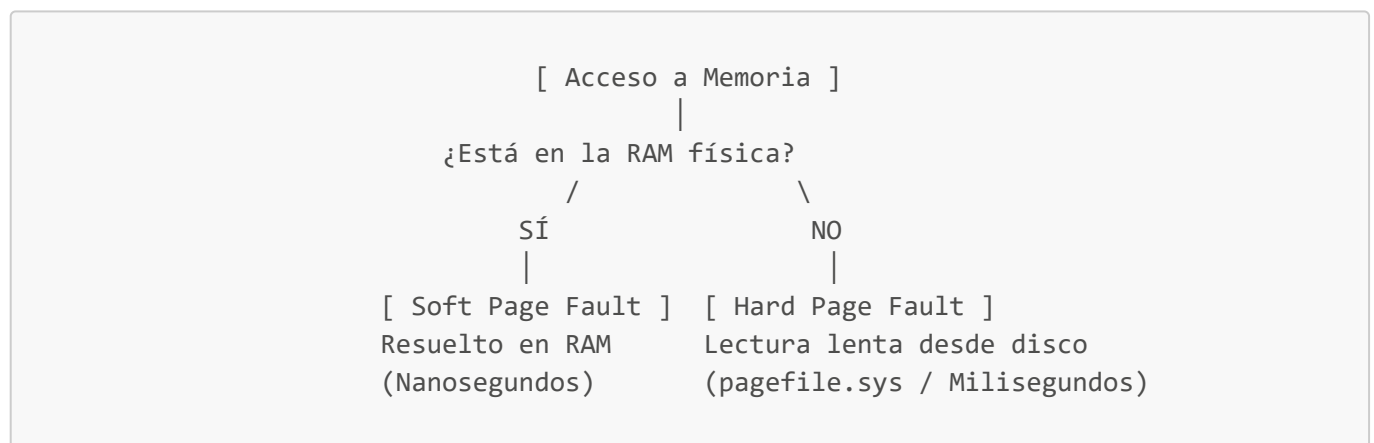
3. Mecanismo de Paginación: Páginas de 4 KB y Page Tables

Windows fragmenta la memoria en bloques estándar de **4 Kilobytes (4096 bytes)**:

- **Página Virtual (*Virtual Page*):** Bloque de 4 KB visto por la aplicación.
- **Marco de Página (*Page Frame*):** Bloque de 4 KB en la memoria RAM física.
- **Tabla de Páginas (*Page Table*):** Estructura del Kernel que mapea dónde reside cada página virtual. Si la página está en RAM, tiene su bit de presencia en **1**. Si fue enviada a disco por falta de espacio, su bit es **0** y apunta al archivo `pagefile.sys`.

4. Fallos de Página (*Page Faults*): Suaves vs. Duros

Cuando un hilo intenta acceder a una página que tiene el bit de presencia en **0**, el procesador dispara una interrupción de **Fallo de Página**:



1. **Fallo de Página Suave (*Soft Page Fault*):** La página requerida ya se encuentra en la RAM física (por ejemplo, en la lista *En espera / Standby* o en la memoria compartida de una librería DLL), pero no estaba vinculada activamente a la tabla del proceso. La MMU la reconecta en **nanosegundos**.
2. **Fallo de Página Duro (*Hard Page Fault / Error de hardware*):** La página no está en la memoria RAM. El Kernel debe suspender el hilo, ordenar al controlador de disco que lea los 4 KB desde el archivo `pagefile.sys`, cargarlos en un marco libre de RAM y actualizar la tabla de páginas. Demora **milisegundos** (miles de veces más lento).
 - **Thrashing / Hiperpaginación:** Ocurre cuando el sistema no tiene suficiente RAM para los conjuntos de trabajo de los programas activos; el equipo pasa el 99% de su tiempo leyendo y escribiendo en el `pagefile.sys` en lugar de ejecutar código útil.

5. Los 4 Estados de la Memoria Física en `resmon.exe`

Al abrir el Monitor de Recursos (**resmon.exe**), la barra inferior muestra:

1. **En uso (Verde):** Código y datos activos de aplicaciones y el sistema.
2. **Modificada (Naranja):** Páginas cuyos datos cambiaron y deben guardarse en disco antes de ser reasignadas.
3. **En espera (Standby - Azul): Caché inteligente.** Datos de programas cerrados o archivos abiertos que Windows conserva en RAM. Si el usuario vuelve a abrir esa aplicación, arranca al instante. Si otra aplicación exige RAM urgente, Windows borra esta caché en microsegundos y la cede.
4. **Libre (Celeste):** Memoria sin inicializar ni utilizar.

Conclusión Didáctica: "Memoria RAM libre es memoria desperdiciada". Un sistema eficiente mantiene casi toda su memoria entre *En uso* y *En espera*.

6. Límite de Compromiso (*Commit Limit*) y Fugas de Memoria

- **Working Set (Espacio de Trabajo):** La cantidad de memoria RAM física real que un proceso tiene cargada en este instante.
- **Commit Charge (Memoria Confirmada):** La memoria virtual total que el proceso ha reservado y que Windows garantizó que respaldará si se escribe en ella.
- **Commit Limit (Límite del Sistema):** $\text{Commit Limit} = \text{RAM Física Instalada} + \text{Tamaño de pagefile.sys}$ Si el *Commit Charge* acumulado de todos los programas alcanza el *Commit Limit*, Windows no puede asignar más memoria virtual y lanza la advertencia: "El equipo tiene poca memoria".
- **Fuga de Memoria (Memory Leak):** Error de programación donde una aplicación solicita memoria continuamente al sistema operativo pero olvida liberarla cuando ya no la necesita, haciendo crecer su *Commit Size* indefinidamente hasta agotar el límite del sistema.

7. Pools de Memoria del Kernel: Paged Pool vs. Non-Paged Pool

- **Paged Pool (Grupo Paginado):** Estructuras del sistema operativo que pueden ser descargadas al archivo **pagefile.sys** si se requiere RAM para aplicaciones de usuario.
- **Non-Paged Pool (Grupo No Paginado):** Memoria **sagrada** del Kernel. Contiene rutinas de interrupción (ISR/DPC), controladores de hardware y el planificador. **Jamás puede ser paginada a disco**, porque si el Kernel sufriera un *Page Fault* mientras atiende una interrupción crítica del disco o del reloj, el equipo entraría en pantalla azul (BSOD).